

ULTRON

Autonomous Coding Harness
Gap Closure & Architecture Upgrade Plan

Version 1.0 • 23 September 2026

Implementation handoff document for the Ultron coding AI

Purpose. This document is the engineering source of truth for the next Ultron architecture upgrade. The coding AI should read this document before changing architecture. It defines the current state, target state, migration order, module responsibilities, contracts, validation gates, and non-goals.

Critical rule: Do not invent existing components. Do not assume a scaffold is implemented. Inspect the repository first, reconcile this plan against actual code, then implement one phase at a time.

P0	Architecture truth audit	Repository and documentation agree on what exists
P0	Context + repository intelligence	Large repositories become navigable within bounded context
P0	Event/state runtime	Long-running tasks become replayable and recoverable
P0	Verification + repair	Edit → test → diagnose → repair becomes a first-class loop
P1	Git checkpoints	Every autonomous change is recoverable
P1	Execution backends	Host/sandbox/container become interchangeable
P1	Tool contracts + concurrency	Typed, observable, safe tool execution
P2	Delegation/subagents	Complex work can be decomposed without destabilizing the core
P2	Observability + benchmark suite	Capability is measured rather than assumed

1. Current Ultron: the truth we are starting from

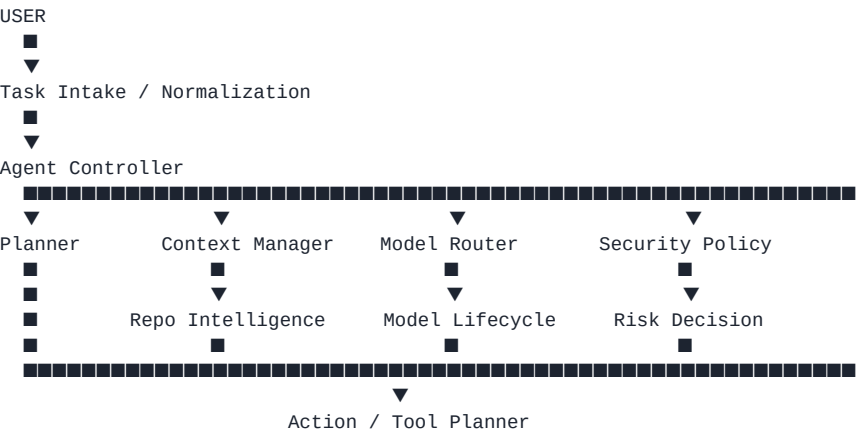
The supplied project context describes Ultron as a local-first, permission-gated, tool-using Python CLI. The snapshot contains a mature set of capabilities: a ReAct agent, 38 registered tools, security gating, memory/knowledge graph, retrieval, planning, parallel tool execution, structured-output validation, resource monitoring, API schema learning, debugging context, multimodal scaffolding, and a responsive CLI.

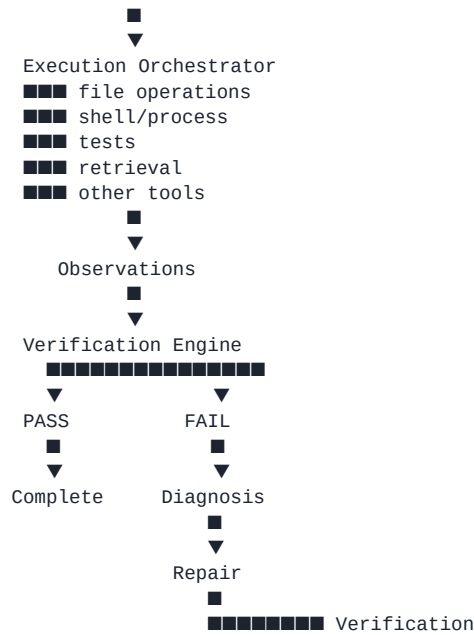
Important repository inconsistency: the handoff context describes some components as implemented while the supplied source snapshot still contains scaffolds. For example, `core/intelligence/model_catalog.py` is a placeholder in the snapshot, and `agents/orchestrator.py`, `codeact.py`, `operative.py`, and `monitor.py` are explicitly scaffold files. The plan therefore starts with a truth audit rather than blindly assuming either the documentation or previous conversation state is correct.

SimpleAgent	Implemented	Keep; route through common runtime contracts
ReActAgent	Implemented	Keep; refactor onto event/state/tool abstractions
Tool registry	Implemented	Keep; add typed contracts + lifecycle events
Security boundary	Implemented	Keep; make it policy-driven and action-aware
Planning/preflight	Implemented	Promote from tools into runtime planning subsystem
Parallel tools	Implemented	Promote to dependency-aware executor
Memory graph	Implemented	Separate long-term memory from task context
Retrieval	Implemented	Extend into repository intelligence
Debug context	Implemented	Connect to verification/repair loop
Model catalog/router	Needs reconciliation	Audit current branch before changing
llama.cpp backend	Scaffold in snapshot	Implement behind BaseEngine
Orchestrator	Scaffold	Do not implement until core runtime is stable
Sandbox backends	Scaffold	Implement after execution abstraction exists
MCP / voice / web UI / desktop UI	Scaffold	Out of scope for core gap closure

2. Target architecture

The target is not 'OpenHands but smaller'. Ultron should combine the strongest architectural lessons from mature coding harnesses with its local-first, hardware-aware model routing and explicit security boundary.





Cross-cutting infrastructure:
 Event Store • Task State • Git Checkpoints • Context/Token Budget
 Execution Backend • Audit Trail • Metrics • Replay

2.1 Architectural invariants

- One authoritative TaskState owns task lifecycle. No subsystem invents a second hidden task state.
- Every meaningful agent action produces a typed event or an auditable record.
- Tools are capabilities, not workflow logic. Planning, verification, retry, and repair belong above tools.
- The model may propose an action; only the runtime can authorize and execute it.
- Security is evaluated on canonicalized actions, not raw model text.
- Context is assembled by a dedicated ContextManager under a token budget; the agent never blindly sends the entire history.
- Repository retrieval is deterministic/reproducible enough to debug: every retrieved context item has a reason/source.
- Autonomous modification must be recoverable through checkpoints or Git.
- Completion is decided by verification evidence and TaskState, never by the model saying 'done'.
- No phase may weaken existing security behavior or silently broaden file/network permissions.

3. Phase 0 — Architecture Truth Audit

Goal: Establish a machine-verifiable baseline before any redesign.

Primary modules: Repository tree, implementation status, tests, docs, configs, model backends.

Implementation work:

- Run the complete existing test suite, lint, stress harness, and resize E2E where available.
- Inventory every module and classify it as implemented, partial, scaffold, dead/unused, or unknown.
- Compare PROJECT_CONTEXT.md, README.md, docs/, and source behavior; record discrepancies.
- Trace the actual chat path from CLI input to model call to tool execution to response.
- Trace security gating for every state-changing tool.

- Trace TaskState/PlanStep behavior and identify duplicated or hidden state.
- Record the actual model backend(s) available on the target branch. Do not assume llama.cpp is live because a file exists.
- Produce a baseline report containing test count, lint status, active agents, active tools, and known failures.

Required validation:

- No implementation changes except audit instrumentation or fixes required to make the baseline runnable.
- Baseline report is committed to docs/ultron_baseline.md.
- Every planned change in later phases names an existing file or a new file explicitly.

Exit criteria: A clean baseline is not required; an accurate baseline is required.

Known risk: Documentation may be stale. Source code and executable tests are the authority for behavior.

4. Phase 1 — Canonical TaskState + Event Model

Goal: Create one durable runtime state model and an event stream that can explain everything the agent did.

Primary modules:

 core/types.py, new core/runtime/, core/state.py, agents/base.py, tools/registry.py, security/audit.py.

Implementation work:

- Define typed events: UserRequest, TaskCreated, PlanCreated, ModelSelected, ContextBuilt, ToolRequested, ToolAuthorized, ToolStarted, ToolCompleted, ToolFailed, VerificationStarted, VerificationPassed, VerificationFailed, RepairStarted, RepairCompleted, CheckpointCreated, TaskCompleted, TaskFailed, TaskCancelled.
- Define TaskState fields: task_id, goal, status, plan, current_step, attempts, requirements, evidence, checkpoints, errors, timestamps, model decisions.
- Make event IDs, task IDs, and action IDs stable and unique.
- Ensure state transitions are explicit and validated.
- Persist events using a simple append-only JSONL or SQLite backend first; abstract the storage interface.
- Make the runtime replayable: rebuilding TaskState from events must reproduce the same state.
- Do not put model reasoning text into security audit records unless explicitly needed; avoid secret leakage.

Required validation:

- Unit tests for every state transition.
- Replay test: execute task → persist events → rebuild state → compare state.
- Failure/retry/cancel tests.
- Backward compatibility tests for current ReAct and SimpleAgent behavior.

Exit criteria: A task can be stopped, restarted, inspected, and replayed without losing its authoritative state.

Known risk: Do not introduce a database migration dependency unless the existing persistence layer requires it.

5. Phase 2 — Context Manager + Token Budgeting

Goal: Stop context-window failures from being accidental runtime failures.

Primary modules: New core/context/, agents/react.py, prompt_assembly.py, types/history handling.

Implementation work:

- Create ContextManager with a hard token/character budget.
- Separate static system instructions, task state, recent conversation, repository context, tool observations, verification evidence, and long-term memory.
- Implement deterministic context priority: current task > active plan step > latest failure > relevant repository context > recent dialogue > older history.
- Implement condensation/summarization for long histories.
- Track token budget before every model call.
- When a context build cannot fit, reduce lower-priority material instead of sending an oversized prompt.
- Record ContextBuilt events containing counts/IDs/estimated sizes, not sensitive raw content.
- Add model-specific context limits to ModelCatalog/ModelRouter.

Required validation:

- Synthetic long conversation test exceeding the model budget.
- Nested tool-output stress test.
- Context condensation preserves task goal, current step, failure evidence, and important decisions.
- No model call exceeds configured context budget.
- Regression test for the previously observed oversized ReAct prompt.

Exit criteria: Long-running tasks remain executable under bounded context.

Known risk: Do not solve context overflow by simply increasing the model context length.

6. Phase 3 — Repository Intelligence / Repo Map

Goal: Give Ultron Aider-class repository awareness without dumping the entire repository into the prompt.

Primary modules: New core/repository/, retrieval.py, context manager, file tools.

Implementation work:

- Build a repository index for files, languages, symbols, imports, references, tests, configuration files, and generated/ignored paths.
- Use tree-sitter or another reliable parser where available; provide graceful fallback for unsupported languages.
- Build symbol and dependency relationships.
- Implement RepoMap generation under a configurable token budget.
- Implement task-aware retrieval using lexical match + symbol match + dependency proximity + recent-change relevance.
- Always return provenance: file, symbol/region, relevance reason.

- Exclude secrets, build artifacts, virtual environments, caches, and Git internals by default.
- Cache indexes and invalidate only affected files on change.

Required validation:

- Small repository accuracy tests.
- Multi-file dependency retrieval tests.
- Large-repository token-budget tests.
- Index invalidation tests after file edits.
- Task-to-relevant-file benchmark with manually verified expected files.

Exit criteria: For a repository-level task, Ultron can identify the relevant files/symbols without requiring the entire repo in context.

Known risk: Do not build a heavyweight semantic vector database before deterministic indexing and retrieval work.

7. Phase 4 — Typed Tool/Action/Observation Runtime

Goal: Turn tool calling into a reliable execution protocol rather than direct function dispatch.

Primary modules: core/tools/, new core/runtime/actions.py, observations.py, executor.py, security boundary.

Implementation work:

- Define ToolDefinition: name, input schema, output schema, risk metadata, read-only metadata, idempotency metadata, timeout, concurrency policy.
- Define ActionRequest and ActionResult/Observation types.
- Canonicalize tool names before security checks.
- Validate model arguments against schemas before execution.
- Normalize malformed tool calls and return structured errors to the model.
- Separate authorization from execution.
- Add timeout/cancellation support.
- Capture stdout/stderr/exit status for process tools as structured observations.
- Keep tool implementations small; move orchestration logic into the runtime.

Required validation:

- Malformed JSON/tool argument tests.
- Unknown tool tests.
- Schema mismatch tests.
- Timeout/cancellation tests.
- Security canonicalization tests.
- Observation serialization/replay tests.

Exit criteria: Every tool call follows the same request → authorize → execute → observe lifecycle.

Known risk: Existing security decisions must remain at least as restrictive as before.

8. Phase 5 — Verification Engine + Autonomous Repair

Goal: Make edit/test/diagnose/repair a first-class control loop.

Primary modules: New core/verification/, core/intelligence/debug_context.py, task planning, ReAct agent.

Implementation work:

- Define VerificationSpec: commands, expected exit codes, expected files, assertions, and completion requirements.
- After modifications, automatically determine whether verification is required.
- Run the smallest relevant verification first; expand to broader tests when necessary.
- Normalize failures into FailureEvidence: command, exit code, stderr, traceback, affected files, likely subsystem.
- Feed FailureEvidence into the debug context and ContextManager.
- Create RepairController with bounded attempts and explicit stop reasons.
- Detect repeated identical failures and prevent infinite loops.
- Completion requires positive verification evidence when the task is marked verifiable.

Required validation:

- Fix a deterministic failing test end-to-end.
- Repair a syntax error.
- Repair a dependency/import error.
- Repair a behavioral assertion failure.
- Repeated failure stops after configured limit.
- Model claiming success without verification does not complete the task.

Exit criteria: Ultron can autonomously perform bounded repair cycles and explain why it stopped.

Known risk: Do not allow the repair loop to modify security policy, credentials, or unrelated files merely to make tests pass.

9. Phase 6 — Git Checkpoints, Rollback, and Change Provenance

Goal: Make autonomous edits recoverable and reviewable.

Primary modules: New core/vcs/, file tools, runtime/task state.

Implementation work:

- Detect repository state before a task.
- Create a task checkpoint before the first autonomous mutation.
- Record modified files and Git diff after each meaningful mutation group.
- Support rollback to task checkpoint.
- Protect unrelated pre-existing user changes; never blindly reset the entire worktree.

- Expose task-scoped diff and rollback commands.
- Record checkpoint IDs in TaskState and events.

Required validation:

- Dirty worktree test.
- Task modifies two files then fails; rollback restores only task changes.
- Successful task retains a clean, inspectable diff.
- Rollback is blocked when ownership of a change is ambiguous.

Exit criteria: Every autonomous coding task has a safe recovery path.

Known risk: Git operations are security-sensitive and must pass the same authorization boundary.

10. Phase 7 — Execution Backends and Sandbox Abstraction

Goal: Separate agent logic from where commands/files actually execute.

Primary modules: New core/execution/, security/sandbox/, command runner, file tools.

Implementation work:

- Define ExecutionBackend interface: run_command, read_file, write_file, cwd, environment snapshot, resource limits, cleanup.
- Implement HostBackend first.
- Implement a sandbox/container backend behind the same interface.
- Make security policy aware of backend and risk tier.
- Add timeout, CPU/memory limits where supported.
- Capture environment snapshot for debugging: OS, Python version, package manager, working directory, relevant dependency state.
- Make workspace root explicit and immutable for a task unless policy permits otherwise.

Required validation:

- Host backend regression suite.
- Sandbox backend contract suite.
- Path confinement tests.
- Process cleanup tests.
- Resource-limit tests where the platform supports them.

Exit criteria: Agent code does not care whether execution happens on host or sandbox.

Known risk: Do not make Docker mandatory for every task; preserve a lightweight host path for trusted local workflows.

11. Phase 8 — Dependency-Aware Parallel Execution

Goal: Upgrade existing parallel tools into a scheduler that understands dependencies and safety.

Primary modules: core/intelligence/parallel_tools.py, new core/runtime/scheduler.py, ToolDefinition.

Implementation work:

- Represent actions as a dependency DAG.
- Run independent read-only actions concurrently.
- Serialize conflicting writes and dependent actions.
- Apply per-action security checks before scheduling.
- Enforce concurrency limits based on CPU/memory forecasts.
- Preserve deterministic event ordering even when execution is concurrent.
- Cancel downstream work when a required dependency fails.

Required validation:

- Independent reads run concurrently.
- Two writes to the same file never run concurrently.
- Write → test ordering is preserved.
- A failed dependency blocks dependent actions.
- Event replay remains deterministic.

Exit criteria: Concurrency improves speed without making behavior nondeterministic or unsafe.

Known risk: Do not parallelize merely because a model emitted multiple tool calls.

12. Phase 9 — Model Router Integration into the Real Runtime

Goal: Make model selection an execution concern rather than a standalone feature.

Primary modules: core/intelligence/model_catalog.py, model_router.py, hardware_aware.py, engine/, runtime.

Implementation work:

- First reconcile the actual branch: if ModelCatalog/Router are still scaffolds, implement them now; if already implemented, test their real integration.
- Define typed ModelProfile: capabilities, context window, coding strength, vision, tool calling, latency, memory estimate, backend, quantization.
- Define ModelSelectionRequest from TaskState + ContextPlan + hardware snapshot.
- Route based on hard constraints first, then deterministic scoring.
- Explicitly support escalation after failed verification or complex repair.
- Prevent alphabetical/tie-breaking behavior from starving a designated primary model.
- Make model loading/unloading observable and resource-aware.
- Allow a condenser/cheap model to be selected separately from the main coding model where beneficial.

Required validation:

- Routing matrix tests.
- Real CLI moderate-task routing test.
- Coding escalation test.

- Context-window constraint test.
- Memory-pressure test on the target Mac hardware.
- Model starvation regression test.

Exit criteria: The router makes correct, explainable decisions inside the actual task runtime.

Known risk: Do not optimize for model choice before ContextManager and TaskState provide reliable inputs.

13. Phase 10 — Delegation/Subagents and Advanced Agent Modes

Goal: Add orchestration only after the single-agent core is reliable.

Primary modules: core/agents/orchestrator.py, codeact.py, operative.py, monitor.py, new delegation runtime.

Implementation work:

- Implement OrchestratorAgent as a planner/delegator, not another independent runtime.
- Define Subtask contract: goal, scope, allowed tools, context budget, success criteria, artifacts.
- Allow Explorer/Researcher/Tester/Reviewer roles to run as bounded child tasks.
- Keep one parent TaskState with child task IDs and result summaries.
- Prevent child agents from escaping parent workspace/security scope.
- Implement Monitor only as supervision over observable task state; it must not mutate blindly.

Required validation:

- Parent/child task lifecycle tests.
- Child failure propagation.
- Security-scope inheritance.
- Context isolation.
- No uncontrolled recursive delegation.

Exit criteria: Complex tasks can be decomposed without compromising state, security, or debuggability.

Known risk: Do not implement a swarm architecture. Start with bounded one-level delegation.

14. Phase 11 — Observability, Replay, and Capability Benchmarks

Goal: Replace 'it seems to work' with measurable evidence.

Primary modules: runtime events, logging, tests, new benchmarks/.

Implementation work:

- Add per-task metrics: duration, model calls, tool calls, retries, failures, context size, tokens if available, resource peak, files changed.
- Build a replay/debug command from persisted events.
- Create a benchmark suite of realistic coding tasks.

- Measure task completion, verification correctness, unnecessary edits, repeated failures, security violations, context overflows, and rollback correctness.
- Add regression fixtures for every historical Ultron failure.
- Run benchmarks locally on the target hardware and record model/backend configuration.

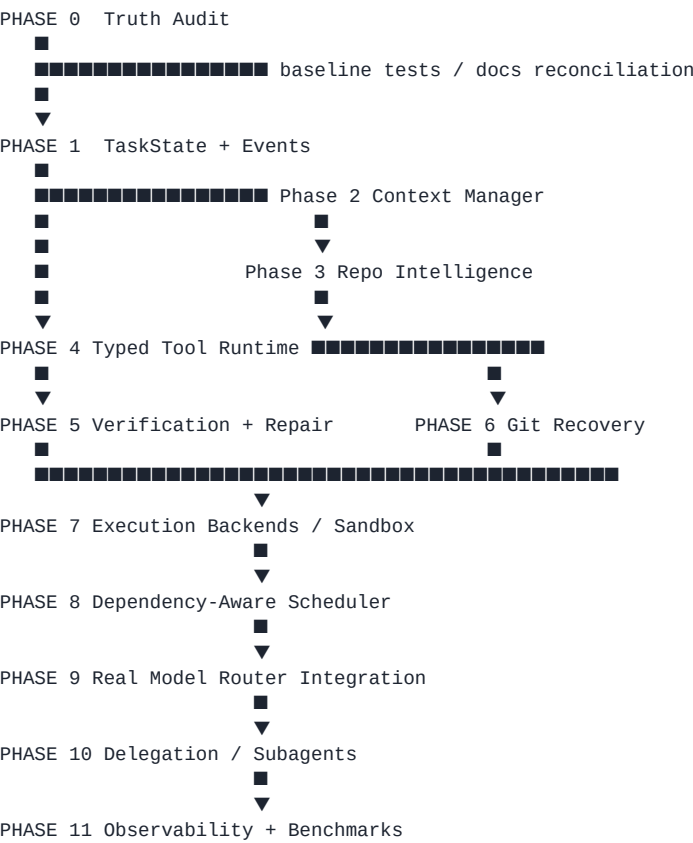
Required validation:

- Replay reproduces state transitions.
- Metrics are generated for every autonomous task.
- Benchmark suite has deterministic fixtures where possible.
- Every regression has a test or benchmark case.

Exit criteria: Future changes can be compared against a stable capability baseline.

Known risk: Do not use a single benchmark score as the definition of intelligence; inspect failure categories.

15. End-to-end dependency roadmap



Recommended implementation rule

Do not skip ahead because a later feature is more exciting. In particular, do not implement subagents before TaskState, events, context, verification, and execution boundaries are stable. Otherwise Ultron will accumulate multiple competing control loops and become harder to debug than the problems it is meant to solve.

M1	Truth audit complete; baseline is recorded and reproducible.
M2	TaskState/event replay works.
M3	Long context is bounded and condensed.
M4	Repo map/retrieval can identify relevant files/symbols.
M5	All tools use typed action/observation lifecycle.
M6	Edit → verify → repair works on real coding tasks.
M7	Git checkpoint/rollback protects autonomous changes.
M8	Host and sandbox execute through one backend contract.
M9	Parallel scheduler respects dependencies and resources.
M10	Model routing is integrated into real task execution.
M11	Bounded delegation works with inherited scope.
M12	Benchmark suite demonstrates capability and regression safety.

16. Proposed module layout

```

src/ultron/
├── core/
│   ├── agents/
│   │   ├── base.py
│   │   ├── react.py
│   │   ├── simple.py
│   │   ├── orchestrator.py
│   │   ├── codeact.py
│   │   ├── operative.py
│   │   └── monitor.py
│   ├── runtime/
│   │   ├── controller.py      # top-level task execution
│   │   ├── state.py          # authoritative TaskState
│   │   ├── events.py         # typed event definitions
│   │   ├── event_store.py    # append/replay
│   │   ├── executor.py       # action lifecycle
│   │   ├── scheduler.py      # dependency-aware concurrency
│   │   ├── checkpoint.py     # task checkpoints
│   │   └── cancellation.py
│   ├── context/
│   │   ├── manager.py        # bounded context assembly
│   │   ├── condenser.py
│   │   ├── budget.py
│   │   └── provenance.py
│   ├── repository/
│   │   ├── index.py
│   │   ├── parser.py
│   │   ├── graph.py
│   │   ├── repo_map.py
│   │   ├── retriever.py
│   │   └── cache.py
│   ├── verification/
│   │   ├── engine.py
│   │   ├── specs.py
│   │   ├── evidence.py
│   │   └── repair.py
│   ├── execution/
│   │   ├── base.py
│   │   ├── host.py
│   │   ├── sandbox.py
│   │   └── resources.py
│   ├── vcs/
│   │   ├── git.py
│   │   └── provenance.py
│   ├── intelligence/
│   │   ├── model_catalog.py
│   │   ├── model_router.py
│   │   ├── hardware_aware.py
│   │   ├── planning.py
│   │   ├── task_planning.py
│   │   └── ...
│   ├── tools/
│   │   ├── registry.py
│   │   └── builtin/...
│   ├── learning/...
│   └── security/
│       ├── boundary.py
│       ├── policy.py
│       ├── guardrails.py
│       ├── audit.py
│       └── sandbox/...
└── tests/

```

Migration principle: do not move every existing file immediately. Introduce new abstractions beside the current implementation, route one execution path through them, then remove duplicate paths after tests prove equivalence.

17. Core contracts the coding AI must preserve

Task completion	Only verification evidence + TaskState may mark a task complete.
Security	No tool bypasses SecurityBoundary. Deny is terminal for that action.
Tool execution	LLM proposes; runtime validates/authorizes; executor runs; observation returns.
Context	Every model call is budget-checked before dispatch.
Repository retrieval	Retrieved context includes provenance/relevance metadata.
Parallelism	Only dependency-independent actions may overlap.
Git	Autonomous mutations have task ownership and recoverability.
Repair	Retry count and repeated-failure detection prevent infinite loops.
Model routing	Hard constraints precede scoring; routing decisions are explainable.
State	One authoritative TaskState; no hidden duplicate lifecycle state.
Persistence	A stopped task can be resumed/replayed from recorded state/events.

18. Validation strategy

Ultron should use a two-layer validation strategy. Layer 1 is deterministic software testing. Layer 2 is model-in-the-loop capability testing through the real CLI/runtime. Neither layer replaces the other.

Unit	state transitions, schemas, router, repo index	Local correctness
Integration	runtime → tool → security → observation	Contract correctness
End-to-end	ultron chat real tasks	Real orchestration
Model-in-loop	repair, repo navigation, multi-file changes	Capability
Stress	long context, many tools, parallel tasks	Reliability
Security	path escape, secret leakage, destructive commands	Safety
Recovery	kill/resume/replay/rollback	Fault tolerance
Benchmark	fixed task corpus	Regression measurement

Required coding-agent benchmark tasks

- Find and fix a failing unit test in an unfamiliar repository.
- Implement a feature spanning 3-6 files and update tests.
- Trace a bug through multiple modules and fix the root cause rather than the symptom.
- Refactor a function while preserving all existing tests.
- Handle a dependency/import failure caused by the local environment.
- Run a targeted test first, then expand verification when required.
- Recover from a deliberately introduced failed edit.
- Perform a task in a dirty Git worktree without destroying unrelated changes.
- Complete a long multi-turn task without exceeding the context budget.

- Reject or request approval for dangerous actions without bypassing policy.

19. Explicit non-goals for this upgrade

- Do not prioritize voice, phone control, desktop UI, web UI, or wake-word features until the coding runtime is reliable.
- Do not create an uncontrolled multi-agent swarm.
- Do not add dozens of tools merely to increase the tool count.
- Do not replace deterministic safeguards with prompt instructions.
- Do not solve context problems by blindly increasing context length.
- Do not add a vector database before deterministic repository indexing/retrieval has been measured and shown insufficient.
- Do not couple the core runtime to one model vendor or one inference backend.
- Do not remove existing tests simply because architecture changed; migrate them to the new contracts.
- Do not declare a phase complete because code exists. A phase is complete only when its exit criteria and regression tests pass.

20. Coding-AI implementation protocol

Put this document in the project root and instruct the coding AI to treat it as the architecture plan. The following protocol is intentionally strict so that an AI coding agent does not hallucinate implementation status or redesign unrelated parts of Ultron.

Before changing code

- Read this PDF plus PROJECT_CONTEXT.md, README.md, AGENTS.md, docs/architecture.md, and the relevant subsystem docs.
- Inspect the actual source files named by the phase.
- Run or inspect the baseline tests relevant to the phase.
- State what currently exists, what is partial, and what is missing.
- If the repository contradicts this document, do not guess. Record the discrepancy and adapt the implementation to the actual code while preserving the target architecture.

While implementing

- Work on one phase at a time.
- Do not implement future-phase features opportunistically.
- Prefer small additive interfaces over large rewrites.
- Preserve existing security behavior.
- Add tests with every behavior change.
- Keep the repository lint-clean.
- Do not fabricate benchmark results, test counts, model capabilities, or implementation status.

After implementing

- Run the phase-specific tests.
- Run the full regression suite.

- Run lint.
- Run stress/E2E tests when applicable.
- For runtime changes, execute at least one real model-in-the-loop CLI scenario.
- Write a concise implementation report listing files changed, tests run, failures, and remaining gaps.
- Only then proceed to the next phase.

Hallucination firewall: If a requested change depends on a class/function/file that does not exist, stop and inspect the repository. Never create a duplicate implementation with a similar name just to satisfy an expected architecture. If an abstraction truly needs to be introduced, add it deliberately and update the architecture map.

21. Final Definition of Done

Ultron's gap-closure program is complete when the system can take a repository-level coding request and execute the following lifecycle reliably:

1. Understand the goal.
2. Inspect repository structure and relevant symbols.
3. Build a bounded, provenance-aware context.
4. Create/validate a task plan.
5. Select an appropriate model using hard constraints + deterministic scoring.
6. Propose typed actions.
7. Apply security policy before execution.
8. Execute safe/approved actions through an execution backend.
9. Record observations/events.
10. Verify the resulting state.
11. Diagnose failures using structured evidence.
12. Repair within bounded attempts.
13. Checkpoint/rollback changes when necessary.
14. Continue until verification proves completion or a clear stop condition is reached.
15. Persist enough state to resume/replay the task.
16. Produce a transparent final report of what changed and what was verified.

The target is not maximum autonomy. The target is bounded, inspectable autonomy: Ultron should be able to do substantial coding work without constant babysitting while remaining recoverable, secure, explainable, and deterministic enough to debug.

End of architecture handoff.